

MP 5149 · UT5 · INTRODUCCIÓ A LES XARXES NEURONALS SUPERVISADES

Del perceptró al perceptró multicapa

Com funciona una neurona artificial, per què una sola no basta, i com una xarxa aprèn a construir ella mateixa les característiques que necessita.

Dos mòduls en paral·lel, dues famílies d'algorismes

MP 5134

Algorismes clàssics

Disseny i avaluació de models

Regressió lineal i logística, k-nearest neighbors, decision trees, random forest, SVM, k-means.

Cada algorisme és **una idea concreta** sobre com separar o ajustar les dades. Els feu servir amb scikit-learn i us centreu a **avaluar-los**.

MP 5149

Xarxes neuronals

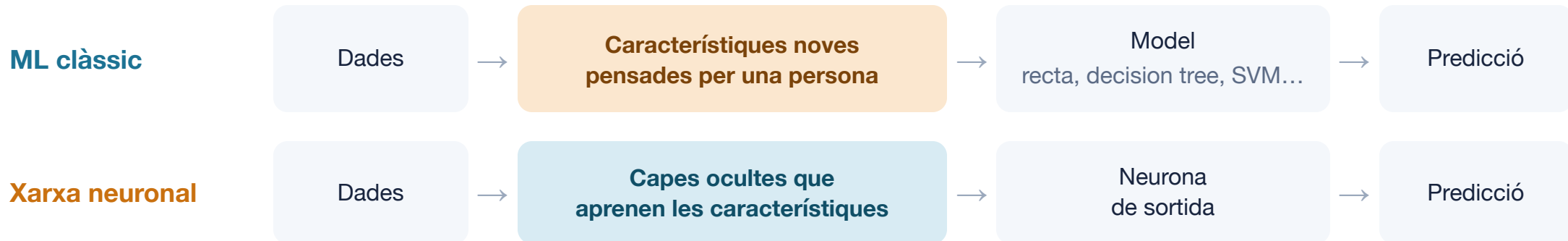
Desenvolupament de components software

Perceptró, perceptró multicapa, mapes autoorganitzatius, xarxes de Hopfield.

Totes es construeixen igual: **moltes neurones molt simples, totes iguals, connectades en capes**. Cap neurona sola fa gran cosa; la potència ve de connectar-ne moltes.

Als dos mòduls es treballa igual: amb scikit-learn, qualsevol model s'entrena amb `fit` i fa prediccions amb `predict`. Aquí no programarem les xarxes per dins ni en farem les fórmules: les **entrenarem amb biblioteques** (scikit-learn i PyTorch) i n'**observarem** el comportament.

Qui fabrica les característiques?



Als algorismes clàssics, les característiques les prepares tu. A les xarxes neuronals, les capes ocultes les aprenen.

Ara sona abstracte. D'aquí a unes quantes diapositives ho veurem passar amb quatre punts.

DUES FAMÍLIES

En què es diferencien

	ALGORISMES CLÀSSICS	XARXES NEURONALS
Què són	Una idea per algorisme: una recta, preguntes encadenades, el marge més ample	Moltes neurones iguals i simples, connectades en capes
Característiques	Les prepara la persona: polinòmiques, transformacions, codificació	Les capes ocultes aprenen representacions noves
Entrenament	Un mètode específic per a cada algorisme; molts donen sempre el mateix resultat	Sempre la mateixa recepta: gradient descent i backpropagation. Depèn de l'atzar inicial
On brillen	Taules de dades estructurades	Imatges, so, text: dades no estructurades i abundants
Cost	Segons, en qualsevol ordinador	Sovint necessiten GPU
Explicació	Molts es poden llegir: coeficients, decision trees	Difícils d'explicar: milers o milions de pesos

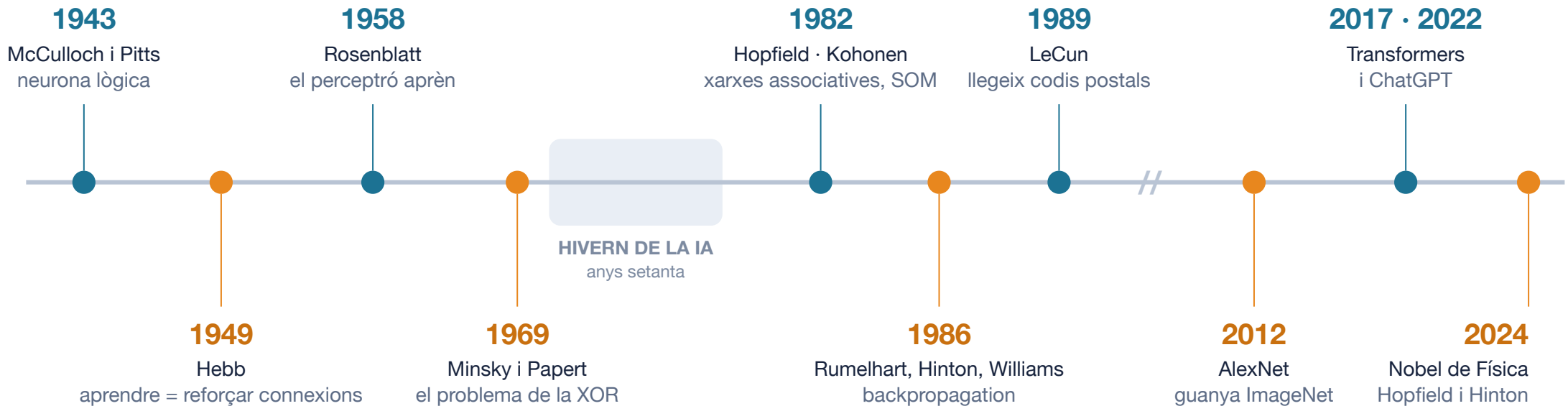
Atenció: **no són sempre millors**. Amb dades tabulars, un random forest o un *gradient boosting* sovint iguala o supera una xarxa, i és molt més barat.

PART 1

Vuitanta anys d'eufòria i d'hiverns

La història de les xarxes neuronals és la d'un problema que ningú sabia resoldre, i de la idea que el va desbloquejar.

Una línia del temps amb alts i baixos



Avui ens quedam entre 1943 i 1986. Hopfield i Kohonen són la UT7 d'aquest mòdul.

De la neurona biològica a l'artificial

BIOLOGIA

Senyals que arriben per les **dendrites**

Força de cada **sinapsi**

El **soma** suma els senyals

Es dispara si supera un **llindar**

Aprendre = reforçar sinapsis

NEURONA ARTIFICIAL

Entrades x_1, x_2, x_3

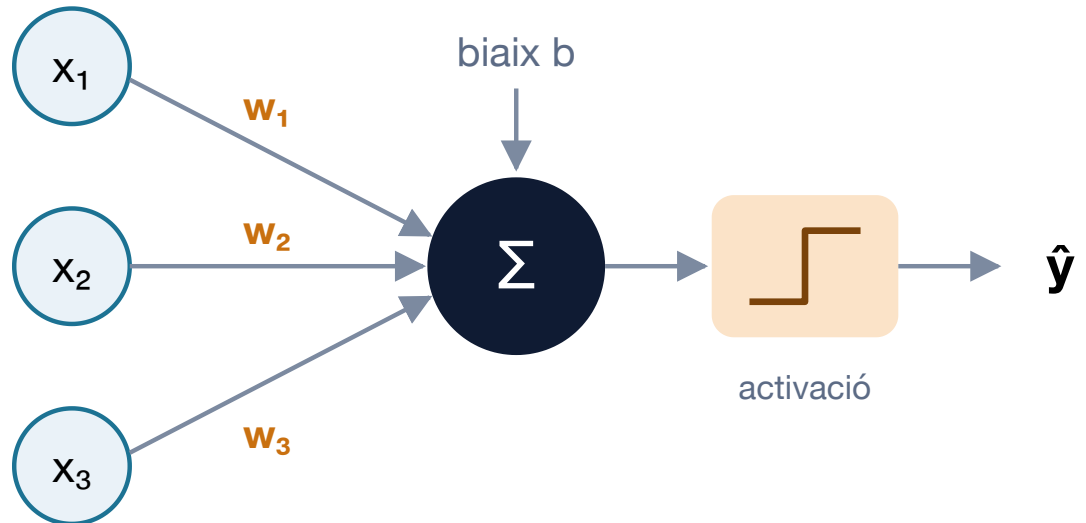
Pesos w_1, w_2, w_3

Suma ponderada

Biaix i funció d'activació

Entrenar = ajustar pesos

El cervell té uns 86.000 milions de neurones. Les xarxes artificials se n'**inspiren**, però no el simulen.



McCulloch i Pitts: una neurona que calcula

Entrades de 0 o 1. La neurona compta quantes n'hi ha d'actives i, si arriba al **llindar**, dispara.

Canviant només el llindar, la mateixa neurona fa portes lògiques diferents. I amb AND, OR i NOT es pot construir **qualsevol càlcul**.

Però no aprèn res. El llindar el tries tu. És programació clàssica amb forma de neurona: `DADES + REGLES → RESPOSTES`

x_1	x_2	AND llindar 2	OR llindar 1
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

Rosenblatt: el perceptró

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + b$$
$$\hat{y} = 1 \text{ si } z \geq 0, \text{ si no } 0$$

Tres novetats respecte de 1943:

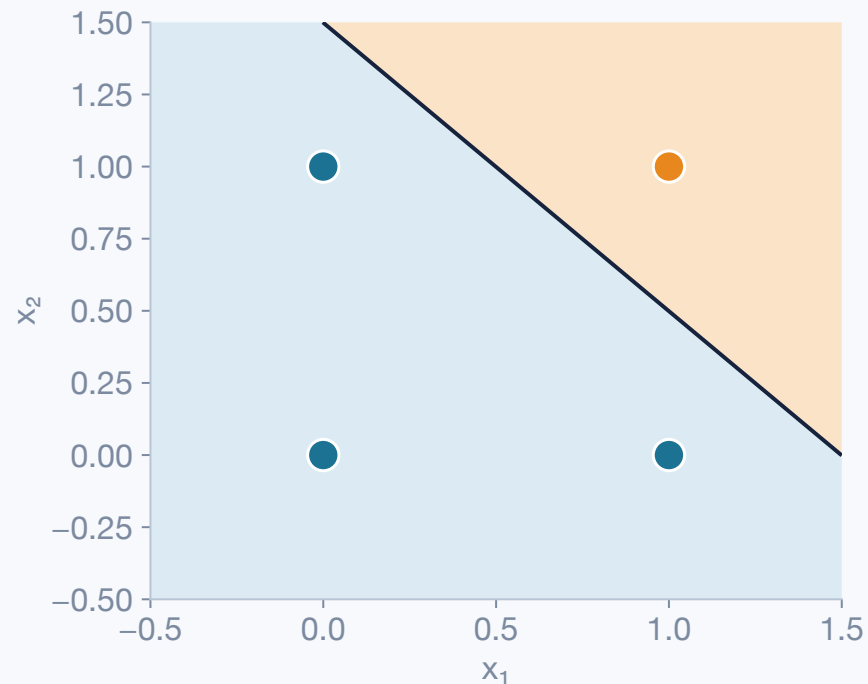
- 1 Les entrades poden ser **qualsevol número**.
- 2 Cada entrada té un **pes**, positiu o negatiu.
- 3 Els pesos **s'aprenen a partir d'exemples**.

El Mark I Perceptron

No era un programa: era una **màquina**. Una «retina» de 400 cèl·lules fotoelèctriques, i els pesos fets amb **potenciòmetres** que uns motors anaven girant mentre aprenia a reconèixer formes.

La premsa de l'època li va atribuir el futur de caminar, parlar i ser conscient de si mateix. Recordau-ho quan arribem a 1969.

Una neurona és una recta



Provem-ho amb pesos $w_1 = 1$, $w_2 = 1$ i biaix $b = -1,5$. La neurona calcula $z = x_1 + x_2 - 1,5$ per a cada punt:

x_1, x_2	z	$z \geq 0?$	$\hat{y}$
0, 0	-1,5	no	0
0, 1	-0,5	no	0
1, 0	-0,5	no	0
1, 1	+0,5	sí	1

És la porta **AND**. La línia negra són els punts on z val exactament 0 ($x_1 + x_2 = 1,5$): per damunt, z és positiu i la neurona diu 1 (zona taronja); per davall, negatiu i diu 0 (zona blava).

Els **pesos** decideixen la inclinació de la recta i el **biaix** la desplaça. Amb $b = -0,5$ la recta baixa i tenim una porta OR. Per tant, **entrenar una neurona és buscar la recta** que deixa cada classe a un costat.

La regla del perceptró

El perceptró comença amb uns pesos qualssevol, és a dir, amb una recta posada a l'atzar. Després repassa les mostres una a una:

per a cada mostra:

1. calcula z i fa la seva predicció $\hat{y}$
2. si encerta $\rightarrow$ no toca res
3. si s'equivoca $\rightarrow$ corregeix els pesos

Una passada per totes les mostres és una **epoch**. Es fan epochs fins que ja no s'equivoca amb cap mostra, o fins a un màxim.

SI...	LA Z ERA...	QUÈ FA
encerta	correcta	no toca res
havia de dir 1 i diu 0	massa baixa	a cada pes li suma una mica de la seva entrada
havia de dir 0 i diu 1	massa alta	a cada pes li resta una mica de la seva entrada

Així, la pròxima vegada la z d'aquesta mostra queda més a prop del costat bo: **la recta es mou una mica** cap a deixar-la ben classificada.

Una entrada que val 0 no ha contribuït a l'error, i el seu pes no es toca. El biaix es corregeix sempre.

Un exemple a mà: la primera epoch de la porta AND

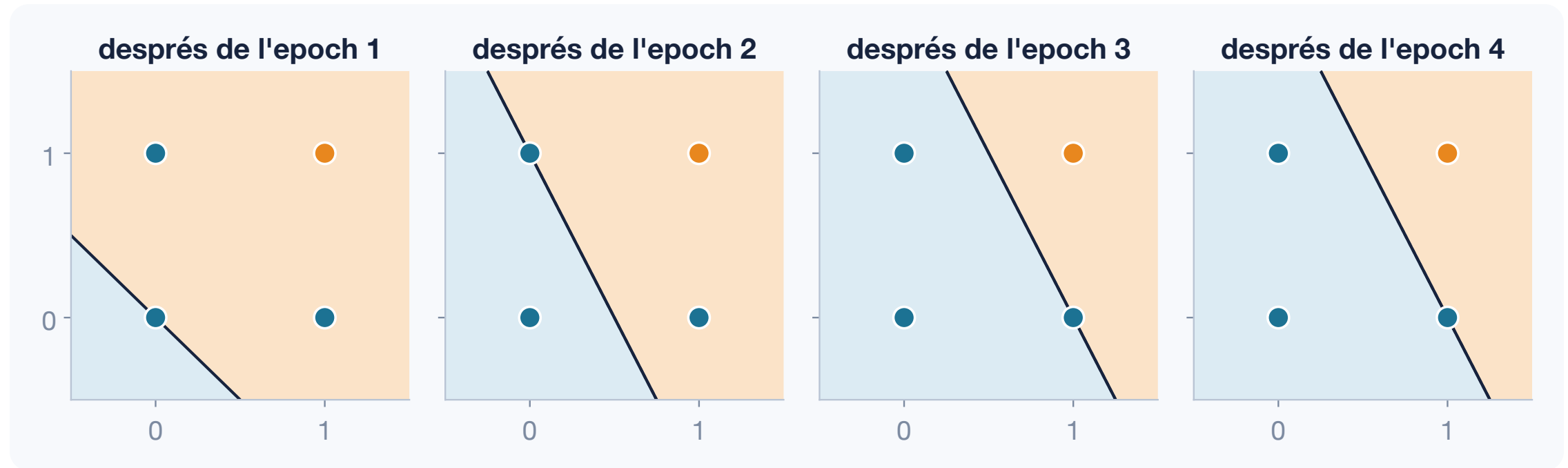
Pesos inicials $w_1 = 0$, $w_2 = 0$, $b = 0$. Cada correcció és de **0,1** (el **learning rate**). Predicció: $\hat{y} = 1$ si $z \geq 0$.

MOSTRA	RESPOSTA	$Z = W_1 \cdot X_1 + W_2 \cdot X_2 + B$	$\hat{Y}$	QUÈ PASSA	PESOS DESPRÉS
(0, 0)	0	$0 + 0 + 0 = 0$	1	Error: havia de dir 0. Com que les entrades valen 0, només baixa el biaix	$w_1 = 0 \cdot w_2 = 0 \cdot b = -0,1$
(0, 1)	0	$0 + 0 - 0,1 = -0,1$	0	Encerta	sense canvis
(1, 0)	0	$0 + 0 - 0,1 = -0,1$	0	Encerta	sense canvis
(1, 1)	1	$0 + 0 - 0,1 = -0,1$	0	Error: havia de dir 1. Pugen els dos pesos i el biaix	$w_1 = 0,1 \cdot w_2 = 0,1 \cdot b = 0$

Dos errors a la primera epoch. A la segona passada es repeteix el mateix amb els pesos nous, i cada error torna a moure la recta una mica.

El **learning rate** (a scikit-learn, `Perceptron(eta=0.1)`) és el primer **hiperparàmetre**: no s'aprèn de les dades, el tries tu. Amb un valor més gran, cada correcció mou la recta més lluny.

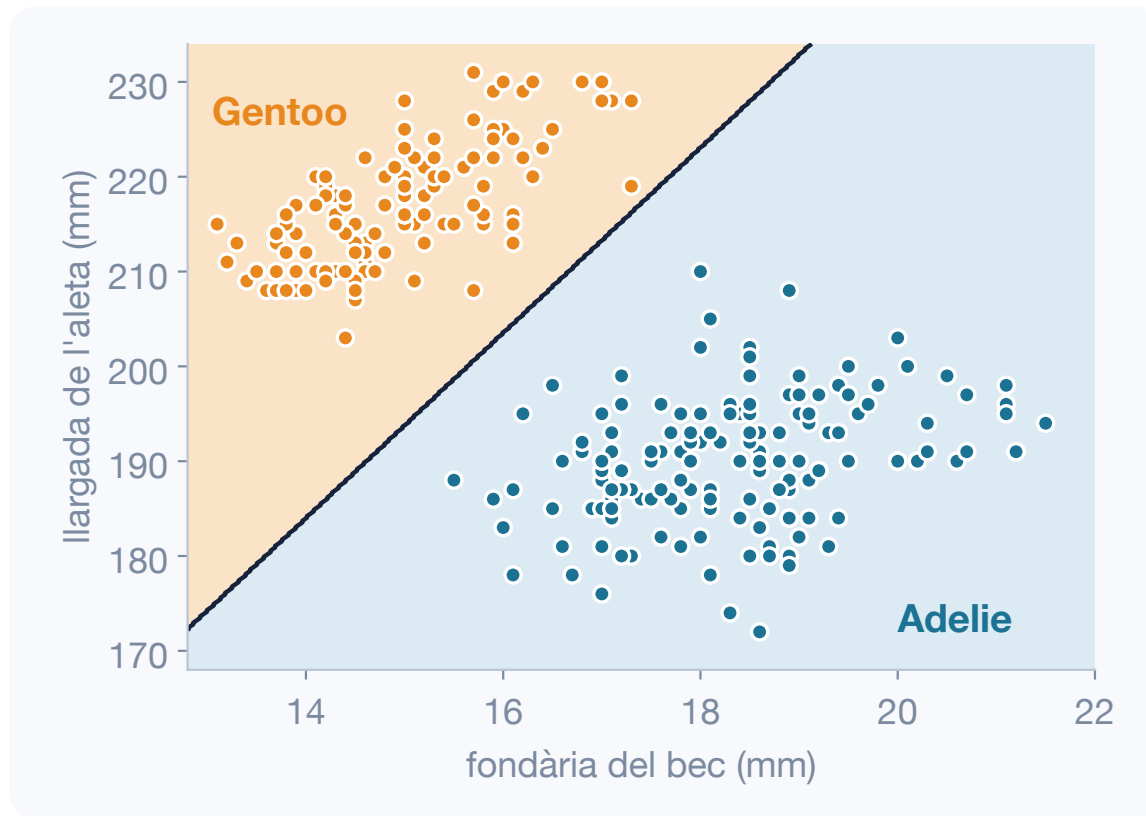
Veure'l aprendre la porta AND



Errors per epoch: `2, 1, 0, 0, 0, 0, 0, 0`. Amb `partial_fit` l'entrenem una epoch cada vegada; a partir de la tercera ja no s'equivoca.

Teorema de convergència: si existeix una recta que separa les classes, el perceptró la troba en un nombre finit de passos. Fixau-vos en el «si».

Adelie o Gentoo, amb una sola neurona



274 pingüins del 5134. Dues mesures. **Encert a test: 100%.**

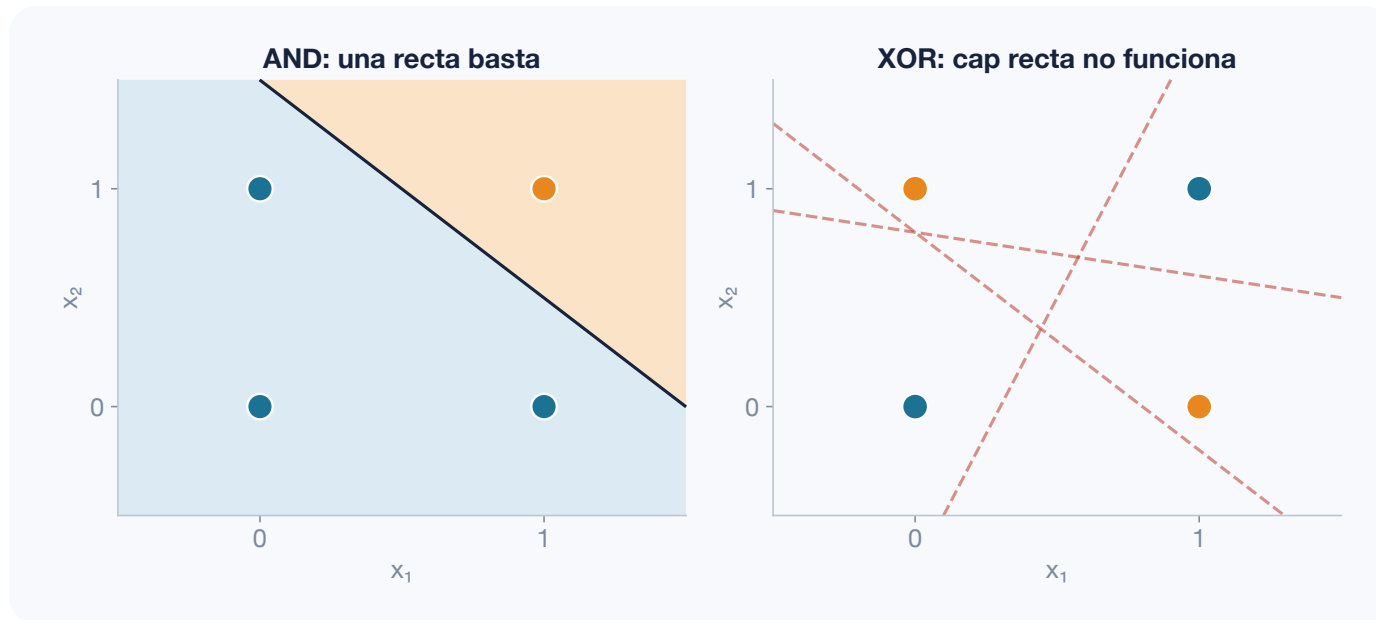
Però mira quant costa aprendre segons l'escala de les entrades. Errors a les primeres epochs:

mesures en mm	98	·	98	·	0	·	0
---------------	----	---	----	---	---	---	---

mesures escalades	1	·	0	·	0	·	0
-------------------	---	---	---	---	---	---	---

L'aleta fa ~200 mm i el bec ~17: les correccions d'un pes són deu vegades les de l'altre. **A les xarxes neuronals, escalar no és opcional:** `StandardScaler` + perceptró en una pipeline.

El mur de la XOR



XOR val 1 quan **una i només una** entrada és 1.

Els punts taronges són a una diagonal i els blaus a l'altra: **no hi ha cap recta** que els separi. El perceptró hi pot estar epochs i epochs.

Minsky i Papert ho van analitzar al llibre *Perceptrons*. La retallada de finançament que va seguir es coneix com el **primer hivern de la IA**.

Com ho resoldria un algorisme clàssic?

x_1	x_2	$x_1 \cdot x_2$	XOR
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Al 5134 la resposta és **fabricar una característica nova**: afegim la columna $x_1 \cdot x_2$ i el mateix perceptró ja ho resol.

Però algú ha hagut de **pensar** que el producte era la columna que calia. Amb quatre punts és fàcil. Amb una foto d'un milió de píxels, quina columna inventes per saber si hi ha un gat?

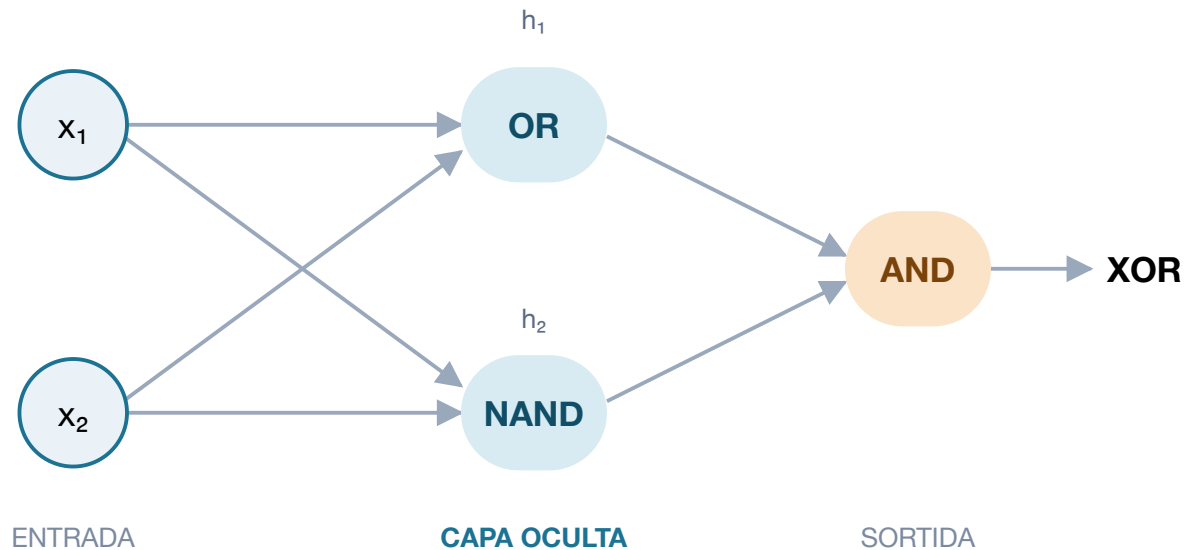
I si la xarxa aprengué sola quines característiques necessita?

PART 2

La solució: afegir capes

Posar neurones entre l'entrada i la sortida. Primer ho farem a mà, per entendre per què funciona. Després, que ho aprengui sola.

La XOR amb una capa oculta

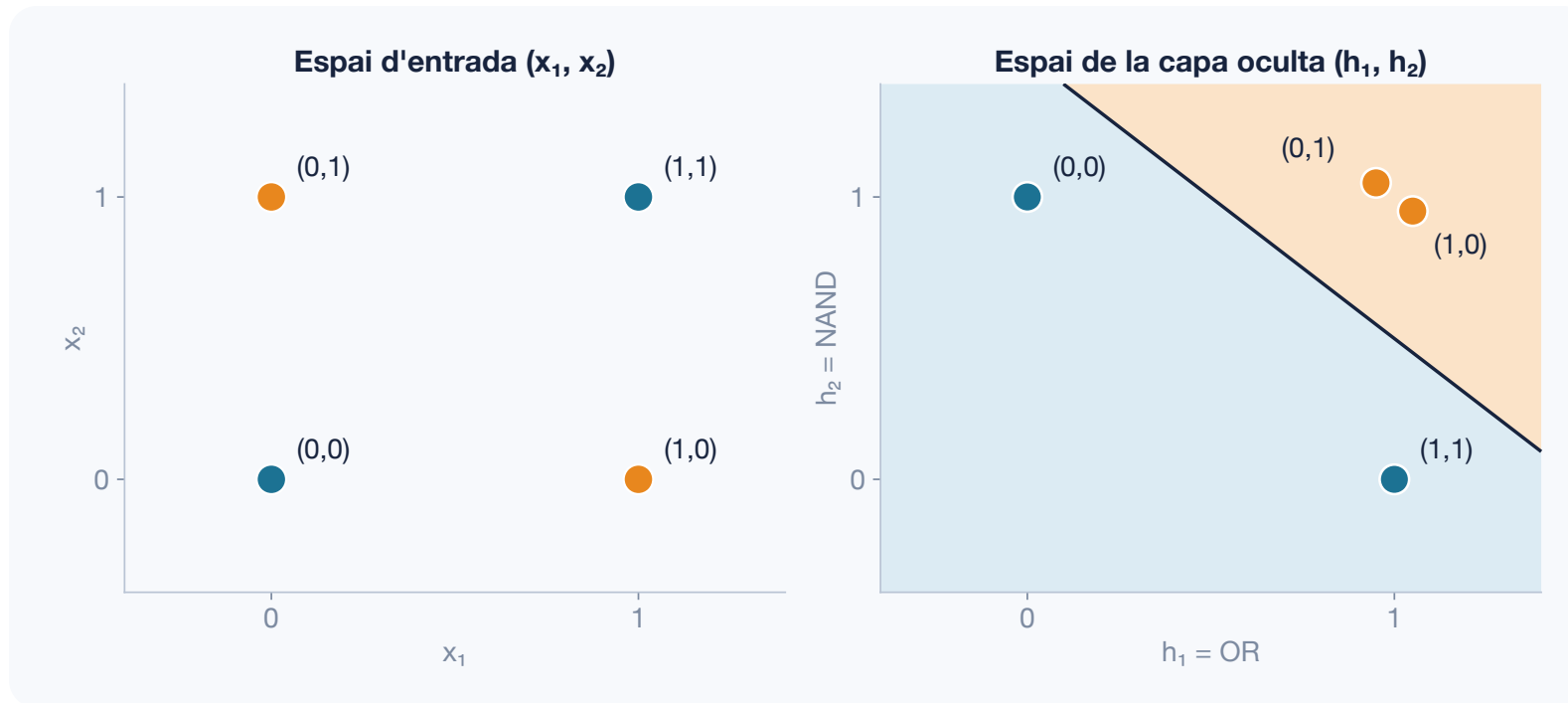


$$\text{XOR} = (x_1 \text{ OR } x_2) \text{ AND NOT } (x_1 \text{ AND } x_2)$$

Cada caixa és un perceptró normal, amb pesos triats a mà. Cap no fa res que un perceptró no sàpiga fer.

Les neurones del mig no es veuen ni a l'entrada ni a la sortida: formen una **capa oculta**. Això ja és un **perceptró multicapa** (MLP).

La capa oculta transforma el problema



A l'esquerra, el problema impossible. A la dreta, **els mateixos quatre punts** vistos per la capa oculta: els taronges cauen junts i una recta ja els separa.

La capa oculta ha **creat característiques noves**, h_1 i h_2 , igual que fèiem a mà amb $x_1 \cdot x_2$. La diferència: en una xarxa entrenada **les tria l'entrenament**.

Per què va costar disset anys?

Saber que les capes funcionen no diu **com trobar-ne els pesos**. Hi havia dos obstacles.

1. A qui tiram la culpa?

La regla del perceptró compara la sortida amb la resposta correcta. Però una neurona oculta **no té resposta correcta**: ningú no li ha dit que havia de fer OR.

Si la xarxa s'equivoca, quina neurona oculta en té la culpa, i quanta?

2. La funció step és plana

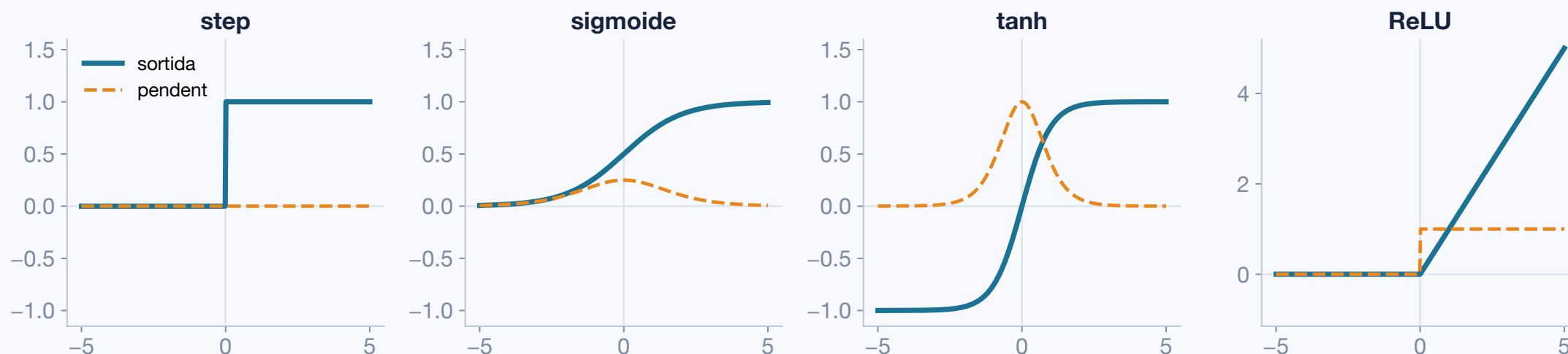
La neurona dona 0 o 1 i res entremig. Si mous un pes una mica, la sortida continua sent la mateixa: **no canvia gens**.

Sense cap canvi, no hi ha pista de cap a on moure el pes per millorar.

La solució té tres peces: **funcions d'activació sense salts, gradient descent i backpropagation**.

Funcions d'activació sense salts

La funció step salta de cop de 0 a 1. Les altres tres fan una **corba sense salts**: si un pes canvia una mica, la sortida també canvia una mica, i això ens diu cap a on moure'l per equivocar-nos menys.



Step. Plana, amb un salt al 0: no dona cap pista. No serveix per entrenar capes.

Sigmoide. De 0 a 1, es llegeix com una probabilitat. A la sortida.

tanh. De -1 a 1, centrada. La farem servir a la capa oculta.

ReLU. L'opció per defecte de les xarxes profundes actuals.

Per al 5134: una neurona amb sigmoide és exactament una regressió logística. El que la converteix en xarxa és apilar-ne moltes.

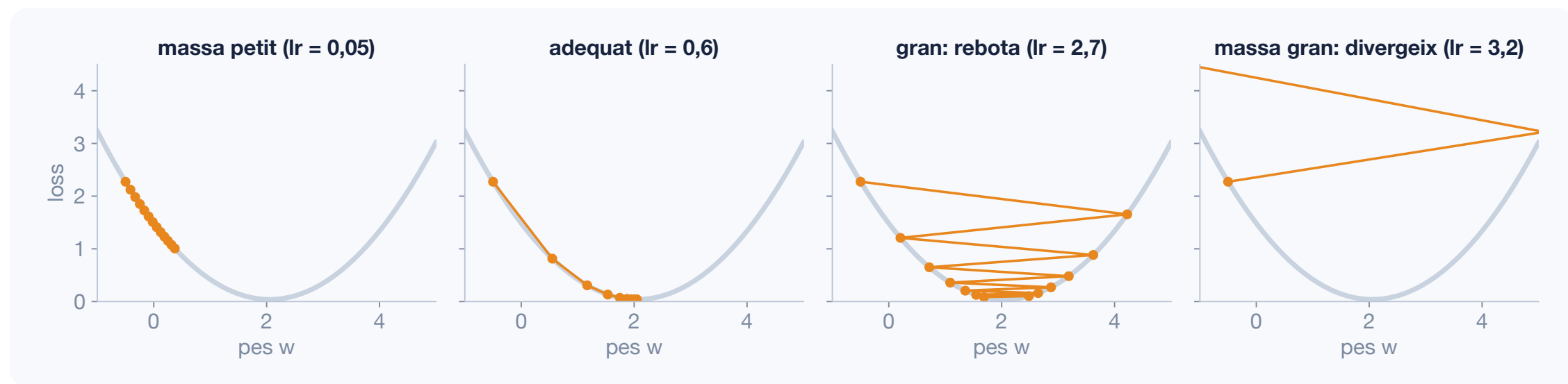
Loss i gradient descent

La **loss** diu com de malament ho fa la xarxa. Al 5134 vau veure que dibuixa una **vall**.

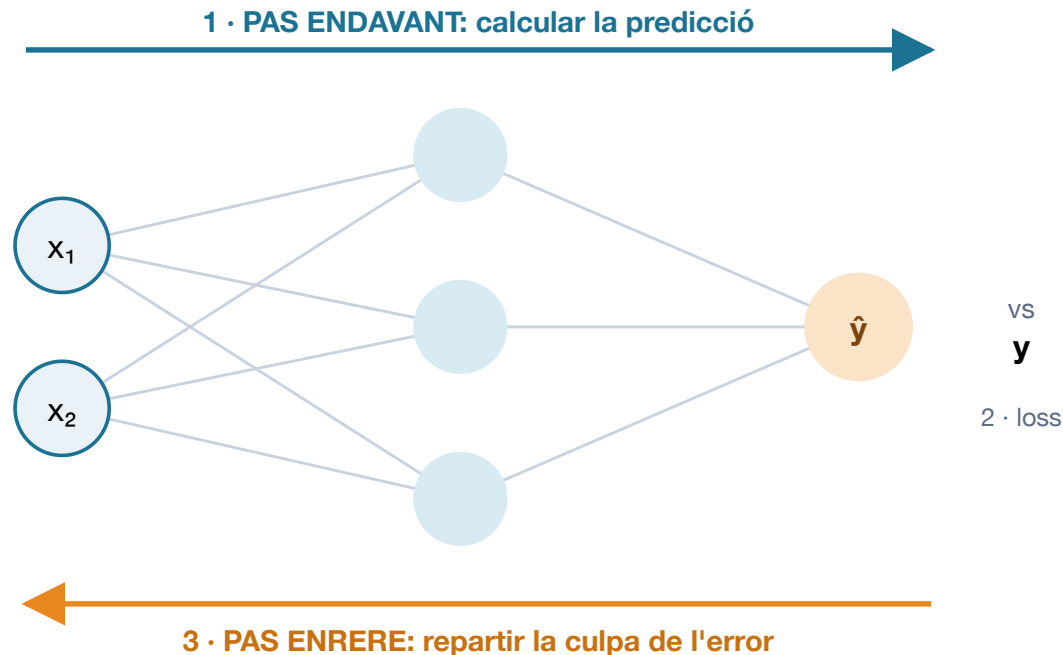
Enmig de la boira, per baixar mires cap a on fa pendent i fas una passa cap avall.

$$w \leftarrow w - \text{learning_rate} \cdot \text{pendent}$$

El conjunt de pendents de tots els pesos és el **gradient**.



Backpropagation: repartir la culpa cap enrere



La sortida sap exactament quant s'ha equivocat: **predicció – resposta**. És la mateixa idea que l'error del perceptró.

Cada neurona oculta en rep una part proporcional a:

- el **pes** que la uneix amb la sortida (si el seu vot pesava molt, té molta culpa)
- el **pendent** de la seva activació (si estava saturada, poca)

4 · Amb la culpa de cada neurona, fem una passa de gradient descent a cada pes.

El bucle d'entrenament

a cada passa d'entrenament:

1. pas endavant → predicció
2. loss → com de malament
3. pas enrere → culpa de cada neurona
4. actualització → una passa avall

```
model = MLPClassifier(hidden_layer_sizes=(3,),  
                      activation="tanh")  
  
for step in range(10):  
    model.partial_fit(X, y, classes=[0, 1])  
    print(model.loss_)    # 0.643 → 0.512
```

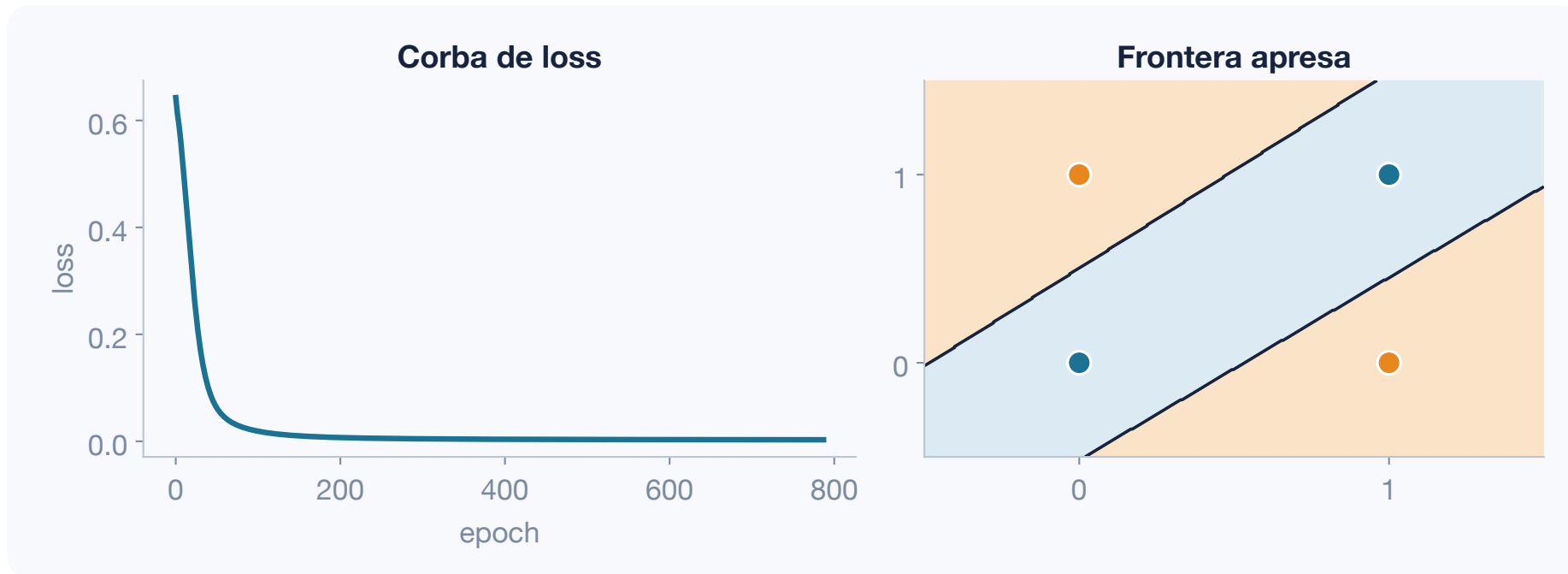
Entrenar una xarxa és **repetir aquests quatre passos milers de vegades**.

Les xarxes que reconeixen imatges o generen text fan aquest mateix bucle, amb més capes, més neurones i més dades.

No ho programem: ho observem.

`MLPClassifier` fa els quatre passos cada vegada que entrena. Nosaltres en mirem el resultat: `loss_curve_`, `coefs_` i la frontera.

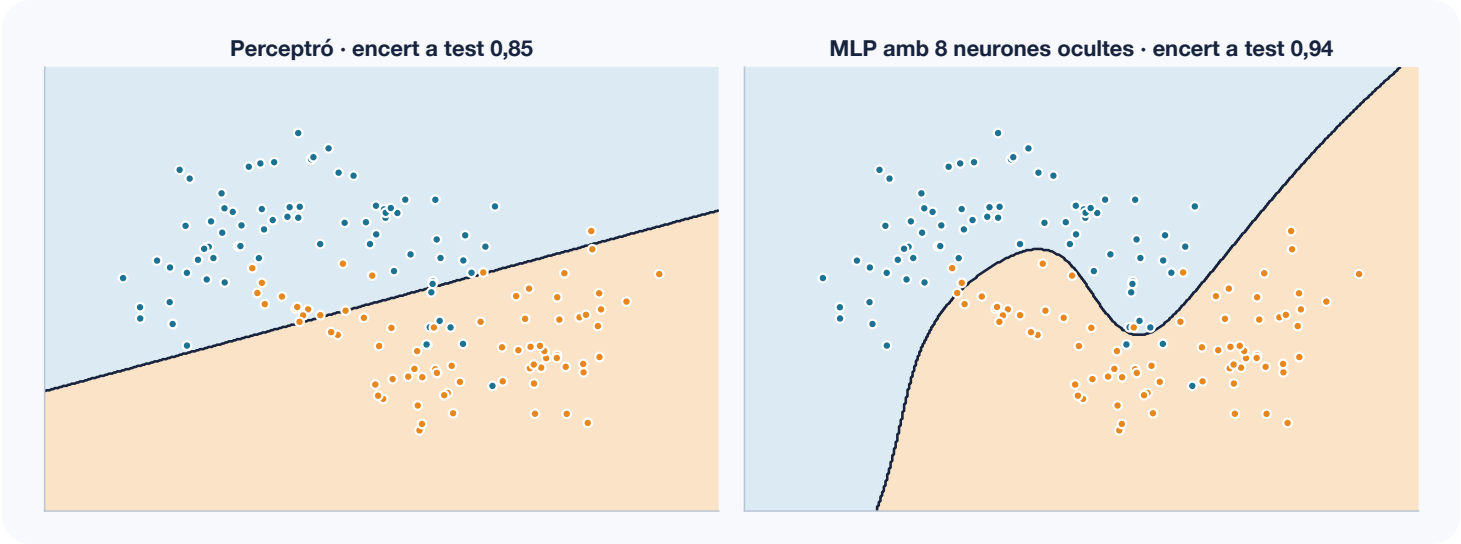
L'MLP aprèn la XOR tot sol



No li hem dit que calien portes OR i NAND ni quins pesos fan falta. Només les quatre files i les respostes.

La corba de loss és el primer gràfic que s'ha de mirar sempre: aquí cau en picat les primeres epochs i després s'aplana a prop de zero. I la frontera **ja no és una recta**: en són dues, que formen una franja.

Les dues llunes

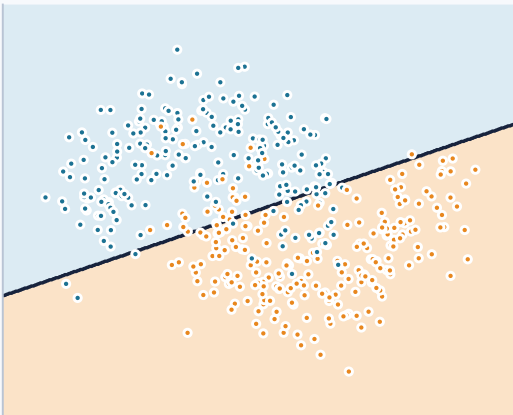


MODEL	ENCERT A TEST
Referència (Dummy)	0,50
Regressió logística	0,86
Perceptron	0,85
MLPClassifier (8, tanh)	0,94
MLPClassifier (16, 16)	0,95

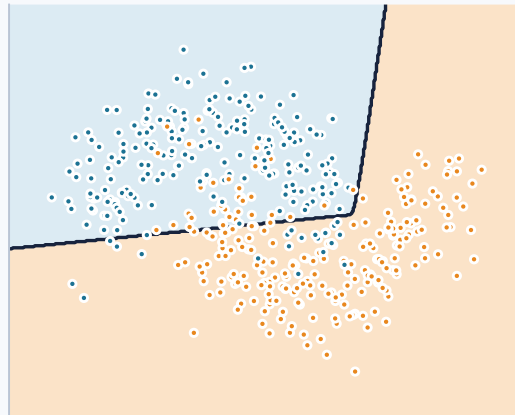
Logística i perceptró són una sola neurona: una recta. No és que siguin dolents, és que el problema no és lineal.

Neurones, learning rate i atzar

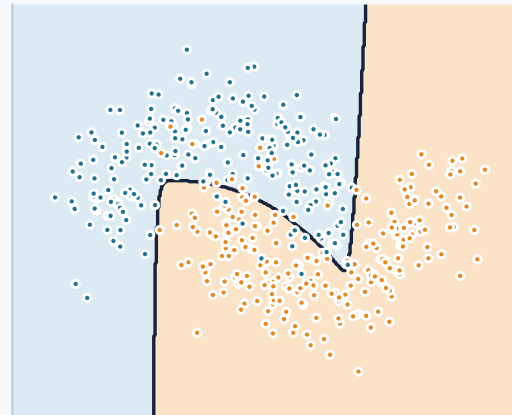
1 neurona · train 0,86 / test 0,86



2 neurones · train 0,87 / test 0,87



4 neurones · train 0,94 / test 0,93



32 neurones · train 0,99 / test 0,92



Neurones ocultes. Amb poques, la frontera no es pot corbar. Amb 32, l'entrenament puja al 0,99 i el test baixa: memoritza el soroll, l'**overfitting**.

Learning rate. Probablement el més important. Massa petit no avança; massa gran rebota o divergeix. Es diagnostica mirant la corba de loss.

La llavor aleatòria. Amb 2 neurones ocultes, la XOR s'encalla o no segons els pesos inicials: un **mínim local**. Una regressió lineal sempre dóna el mateix.

MLPClassifier: cada paràmetre té sentit

```
model = MLPClassifier(  
    hidden_layer_sizes=(16, 16),  
    activation="tanh",  
    learning_rate_init=0.01,  
    batch_size=200,  
    max_iter=2000,  
    random_state=42)  
model.fit(X_train, y_train)  
model.score(X_test, y_test)  
model.loss_curve_
```

PARÀMETRE	QUÈ ÉS
hidden_layer_sizes	capes ocultes i neurones de cada una
activation	funció d'activació de les capes ocultes
learning_rate_init	learning rate
batch_size	mostres per passa (part 3)
max_iter	màxim d'epochs
random_state	llavor dels pesos inicials

La interfície és la de tots els models de scikit-learn: `fit`, `predict` i `score`. **Es pot intercanviar per una regressió logística o un random forest sense tocar la resta del codi.**

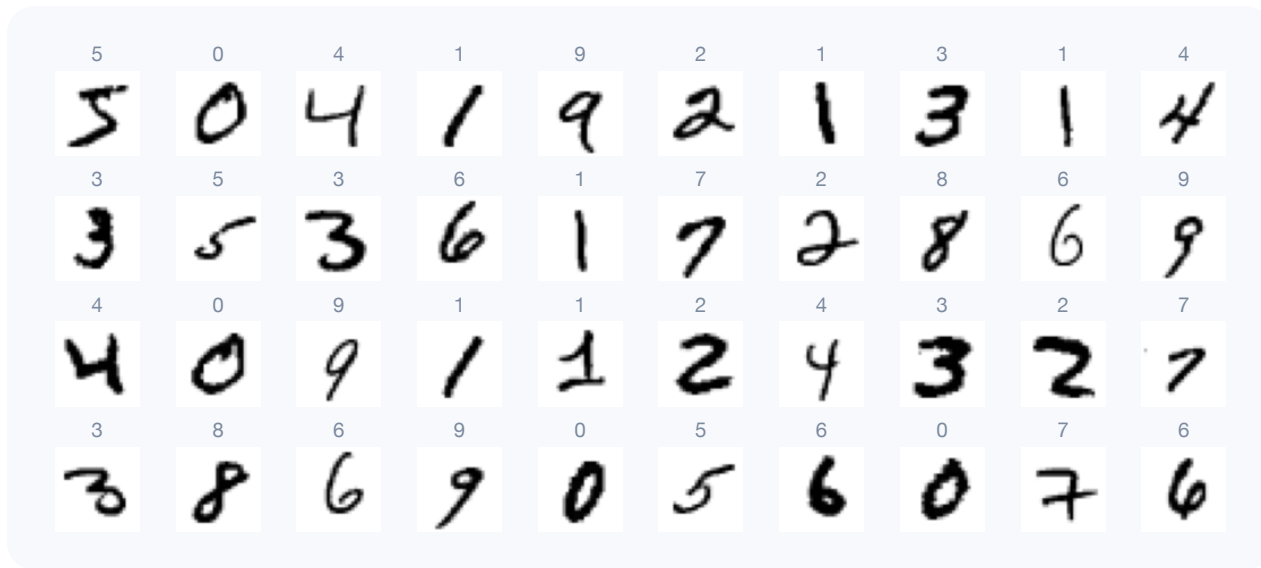
Per defecte fa servir ReLU i l'optimitzador **Adam**, una variant del gradient descent que ajusta la passa de cada pes. La recepta de fons és la mateixa: pas endavant, loss, pas enrere, actualització.

Dades de veritat: 70.000 dígits escrits a mà

Portem l'MLP de les llunes a MNIST. Canviar d'escala obliga a introduir tres idees noves, i ens deixa veure què aprenen de debò les neurones.

3 · MNIST

Una imatge és una fila de 784 números



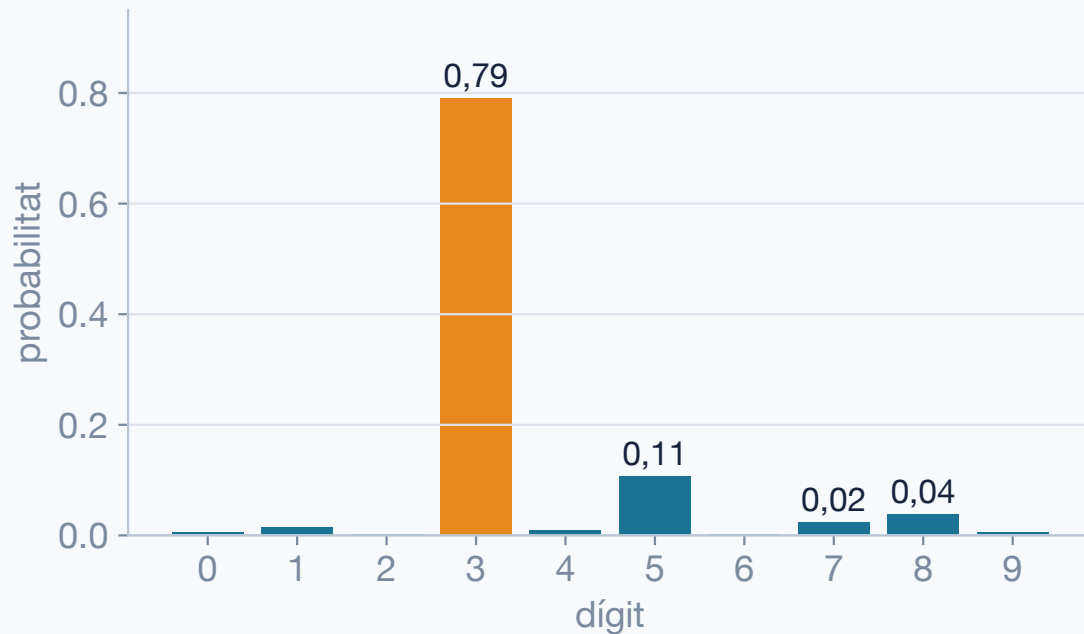
28 × 28 píxels de 0 (paper) a 255 (tinta),
posats en fila: **784 columnes**.

Entrenament	50.000
Validació	10.000
Test segellat	10.000

LeCun i col·laboradors, 1998. Durant anys, *la* prova de referència de la visió per computador. A scikit-learn:

```
fetch_openml("mnist_784") .
```

De 2 classes a 10: softmax

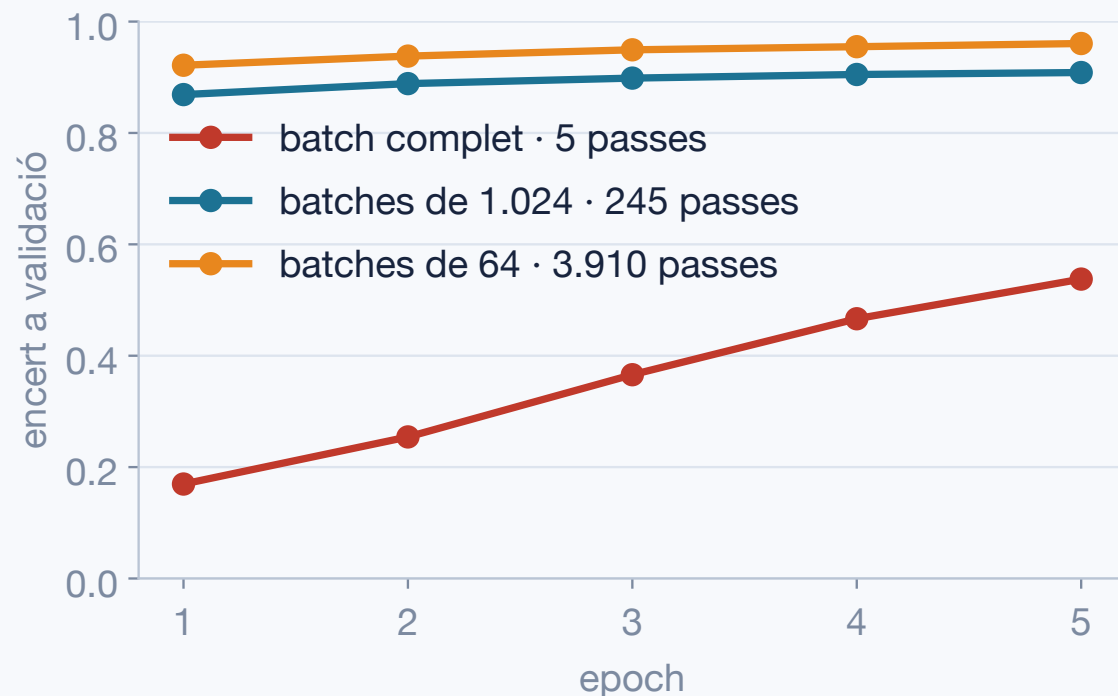


Deu neurones de sortida, una per dígit. La **softmax** converteix les deu puntuacions en deu probabilitats que sumen 1.

La resposta es codifica en **one-hot**: deu posicions, totes a 0 menys la correcta.

MLPClassifier ho fa sol. Amb més de dues classes posa la softmax a la sortida i fa el one-hot per dins; ho diu l'atribut `out_activation_`. La backpropagation és la mateixa.

Mini-batches: moltes passes per epoch



Barrejar, partir en batches de 64 i fer **una passa per batch**.

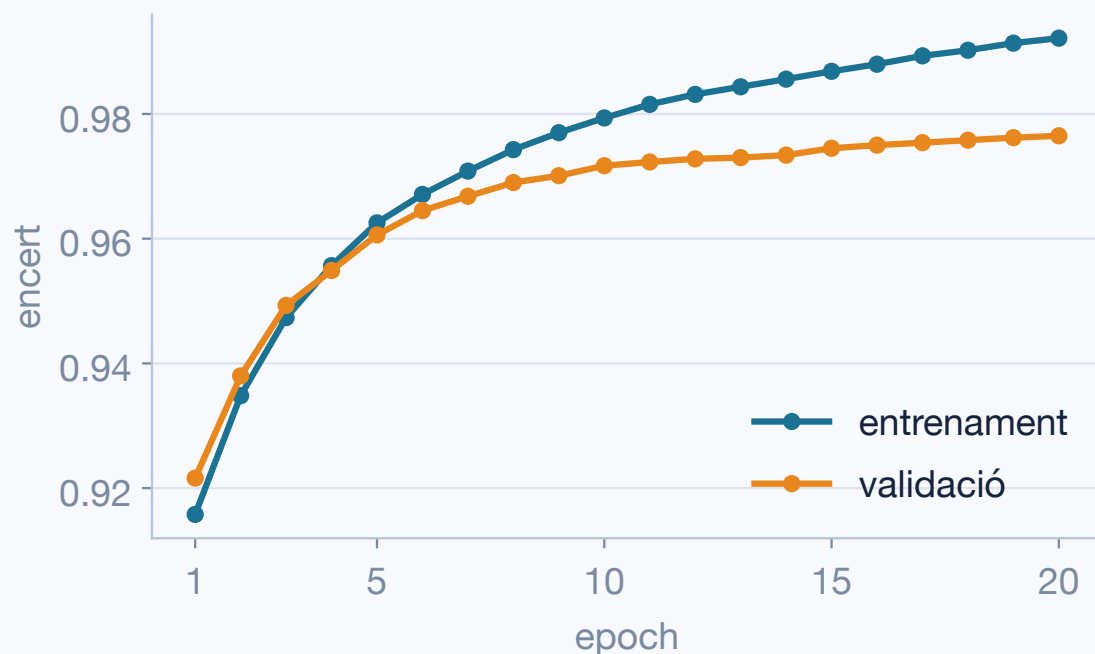
Mateixes dades i mateixes 5 epochs. Amb batches de 64 es fan quasi 800 vegades més passes, però el temps només es multiplica per tres o quatre. A scikit-learn,

```
batch_size
```

Iteració	un batch processat i una actualització
-----------------	--

Epoch	totes les dades: 782 iteracions
--------------	---------------------------------

Validació durant l'entrenament



Xarxa **784-128-10**: més de **100.000 pesos**. La de la XOR en tenia 13.

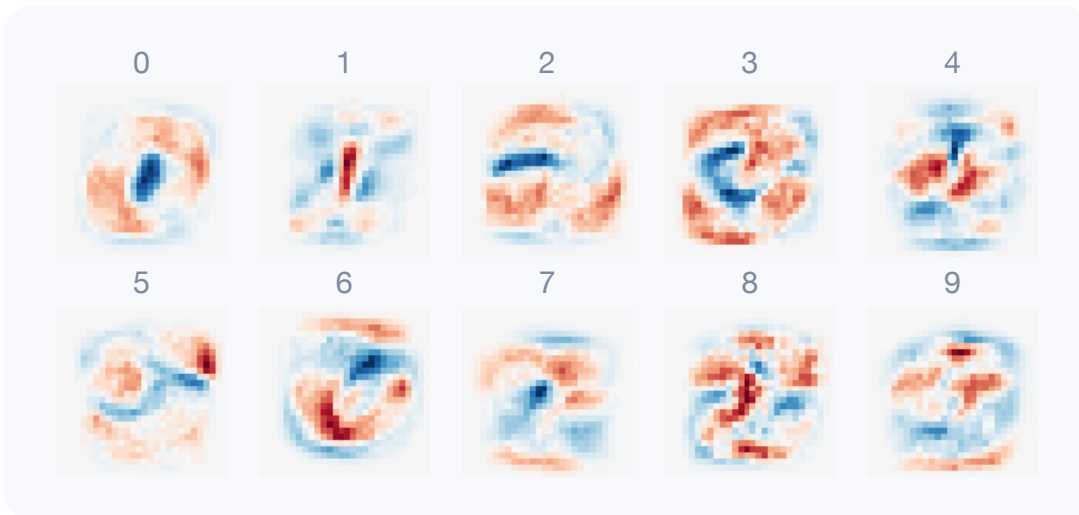
`MLPClassifier` amb tanh i batches de 64, entrenat epoch a epoch amb `partial_fit` : **97,65% a validació** en 20 epochs.

Al principi les dues corbes pugen juntes. Cap al final l'entrenament continua millorant i la validació s'atura: comença l'**overfitting**.

Test segellat, obert al final: **97,61%**. L'estimació de validació era fiable.

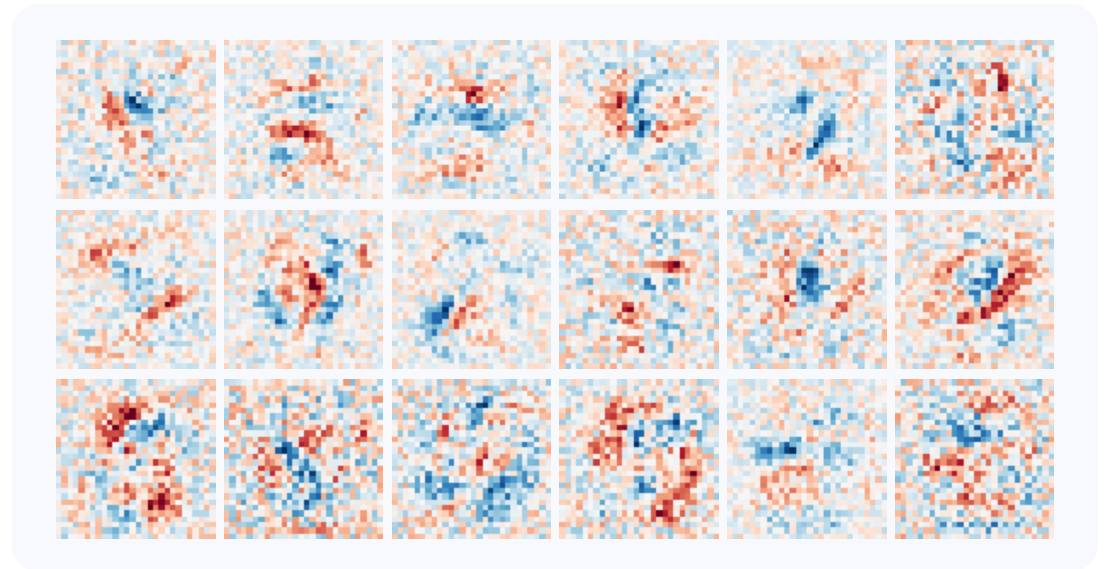
Plantilles contra traços

REGRESSIÓ LOGÍSTICA



Un model lineal aprèn **una plantilla per dígit** i hi compara cada imatge.

NEURONES OCULTES DE L'MLP



Cap no és un dígit: són **traços** en una zona. La sortida combina quins traços hi ha. **Ningú no li ha dit que busqués traços.**

Les característiques apreses serveixen

El mateix model lineal, una regressió logística, amb tres entrades diferents:

784 píxels originals

92,8%

Una frontera lineal sobre píxels no dona per més.

128 característiques **apreses** per la capa oculta

97,9%

Sis vegades menys columnes i molt més encert.

128 característiques d'una xarxa **sense entrenar**

90,9%

El control: passar per neurones no fa màgia.

És la figura de la XOR a gran escala: la capa oculta transforma les dades en una representació on un model lineal ja funciona.

Les xarxes guanyen, però per poc

MODEL	ENCERT A VALIDACIÓ
Referència (Dummy)	10,6%
Regressió logística	92,8%
Random forest (100 decision trees)	97,2%
MLP (128, tanh)	97,65%
MLP per defecte (ReLU, Adam)	97,8%

Un random forest, sense capes ocultes, queda a menys d'un punt.

EL LÍMIT: DESPLAÇAR 3 PÍXELS



Regressió logística	92,8% → 42,2%
Random forest	97,2% → 54,2%
MLP (128, tanh)	97,65% → 53,6%

Per a l'MLP la imatge és una taula: **no sap quins píxels són veïns**. Les xarxes **convolucionals** reutilitzen el mateix detector a totes les posicions, i llavors la distància es fa enorme. Són la **UT6**.

RESUM

Les sis idees que us heu d'endur

- 1 Una **neurona** fa una suma ponderada, hi suma un biaix i aplica una activació. Separa l'espai amb una recta.
- 2 El **perceptró** aprèn corregint els pesos quan s'equivoca, però només resol problemes **linealment separables**.
- 3 Les **capes ocultes** transformen les dades en una representació on el problema sí que és separable.
- 4 Per entrenar capes calen **funcions d'activació sense salts**, una **loss** i el **gradient descent**.
- 5 La **backpropagation** reparteix la culpa de l'error cap enrere per saber com moure cada pes.
- 6 Amb dades reals calen **softmax**, **mini-batches** i **validació**. La capa oculta aprèn característiques, i l'**arquitectura decideix quines** pot aprendre.

Cinc preguntes sense una única resposta

- 1 Si una regressió logística és una neurona, per què diem que les xarxes són una **família diferent**?
- 2 El 1958 es va prometre massa i va venir un hivern. Hi veieu **paral·lelismes amb avui**?
- 3 Una xarxa encerta el 98% però no s'explica; un decision tree encerta el 94% i es llegeix. Quin triaríeu per **concedir un crèdit**? I per **trobar pingüins en fotos aèries**?
- 4 La backpropagation és de 1986 i les xarxes no dominen fins al 2012. **Què faltava**?
- 5 A MNIST, un random forest gairebé iguala l'MLP. Per què, doncs, les xarxes **han guanyat en imatges**?